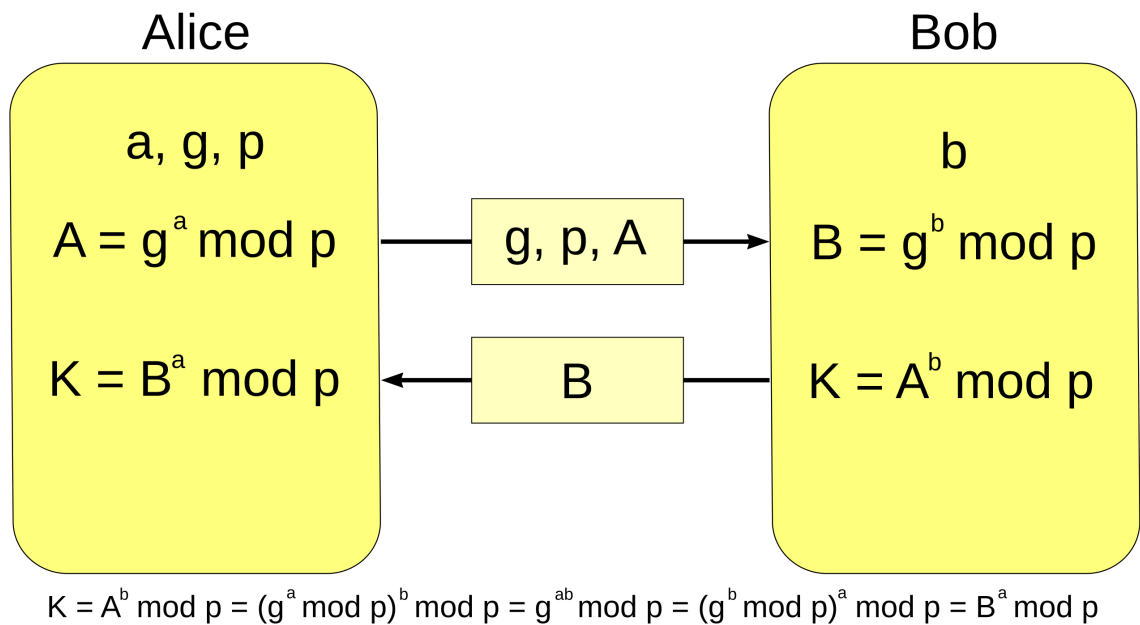




Pour l'exemple, je suis Alice

Importe la librairie "SecureRandom", ...

```
In [1]: import java.security.SecureRandom;
import java.nio.charset.StandardCharsets;
```

## Algorithme de Diffie-Hellman:

→ Génère "a", attention reste secret jamais envoyé, c'est la clef secrète

```
In [2]: BigInteger a;
a = new BigInteger(128, new SecureRandom());
System.out.println("a= "+a);
```

a= 190071901904381649340139200886346599825

→ Génère "p", attention obligatoirement un nombre premier (pour valider le petit théorème de Fermat)

```
In [3]: BigInteger p;
p = BigInteger.probablePrime(128, new SecureRandom());
System.out.println("p =" +p);
```

p =285608035271385635455319471425883031541

→ Génère "g", attention obligatoirement  $1 < g < p$

```
In [4]: BigInteger g;
do{
    g =new BigInteger(128, new SecureRandom());
```

```
System.out.println("g= "+g);
}while(g.compareTo(p)>0);
```

```
g= 319959859672288077984289593482639179395
g= 329880246477583598760116684313753628682
g= 73959535966034343418514599346895011134
```

→ Calculer "A", qui est une partie de la clef publique

```
In [5]: BigInteger A;
        A = g.modPow(a, p);
        System.out.println("A= "+A);
```

```
A= 134780000937803081924627721931212952425
```

## Configuration de la socket @127.0.0.1:5555

```
In [7]: Socket client = new Socket(InetAddress.getByName("127.0.0.1"),5555);
```

→ Récupère les flux de la socket

```
In [8]: InputStream in = client.getInputStream();
        OutputStream out = client.getOutputStream();
```

→ Envoyer "p", "g", "A" à Alice

```
In [9]: out.write(p.toByteArray());
        System.out.println("Client : p = " + p);
        out.write(g.toByteArray());
        System.out.println("Client : g = " + g);
        A = g.modPow(a,p);
        out.write(A.toByteArray());
        System.out.println("Client : A = " + A);
```

```
Client : p = 285608035271385635455319471425883031541
Client : g = 73959535966034343418514599346895011134
Client : A = 134780000937803081924627721931212952425
```

→ Réception la clef publique B de Bob

```
In [12]: byte[] byte_B=new byte[65535];
        int taille=0;
        BigInteger B;
        new Thread()->{
            System.out.println("Attente");
            try{
                taille=in.read(byte_B);
                B= new BigInteger(Arrays.copyOfRange(byte_B, 0, taille));
                System.out.println("Serveur : Clef publique = " + B);
            }catch(Exception e){
```

```
}  
}).start();
```

Attente

→ Calcul de la clef de cryptage K

```
In [14]: BigInteger K;  
         K = B.modPow(a,p);// calcul de la clef secrète  
         byte[] byteArray = K.toByteArray();  
         System.out.println("""  
             Clé privée du cryptage symétrique ----> %s  
             En hexadécimal ----> %s  
             En Utf-8 (si possible) ----> %s  
             """).formatted(K,toHex(byteArray),new String(byteArray, StandardCharsets
```

```
Clé privée du cryptage symétrique ----> 54582858125616579451440626935095642454  
En hexadécimal ----> 291046ADEDAA86B40FFCF3A281E44956  
En Utf-8 (si possible) ----> )?F?????IV
```

→ Fermer les flux et la socket

```
In [11]: in.close();  
         out.close();  
         client.close();
```

## Méthode pour convertir byte[] en hexadécimal

```
In [13]: public static String toHex(byte[] bytes) {  
         StringBuilder sb = new StringBuilder();  
         for (byte b : bytes) {  
             sb.append(String.format("%02X", b));  
         }  
         return sb.toString();  
     }
```